Bayou application and engineering specification

Application flows User stories Data model Security Company operations Efficiency Scalability

Prepared for the product owner, implementation team and operating company. Version 1.0 dated 29 October 2026. Code baseline f3167e8 on codex/ship-ready-mvp. Bayou was previously named KadoLakay; historical repository and integration identifiers retain that name.

Bayou is a trilingual gift registry connecting families and approved local businesses. Owners create and share lists; guests discover a shared list and choose gifts; merchants maintain products and confirmed stock; administrators review businesses. The product must protect money, inventory and private information while remaining practical on mobile connections.

The core registry, merchant catalog, local Google login, QR sharing, shared stock and checkout retry logic exist. The complete application is not yet cleared for launch. New online payments remain disabled. A successful build or a provider preview marked Ready does not prove successful payment, fulfillment, security or deployment readiness.

Status label	Meaning
Implemented	Present in this code baseline. This does not imply a complete hosted browser journey.
Integration pending	Code exists, but credentials, external account eligibility or hosted acceptance remain unverified.
Planned	Required behavior or operating policy that is not implemented in the current product.
Decision required	The owner or operating company must choose a business rule before implementation or activation.

Audience and accountability

The product owner approves scope and commercial decisions. Developers implement and test invariants. The operating company owns merchant review, support, reconciliation and incident response. Merchants own accurate product information and physical counts. Payment providers authorize and process payments; Bayou must verify their evidence before recording success.

Current actors

Guest giver; authenticated registry owner; BUSINESS_OWNER merchant; ADMIN reviewer. A superadmin role, company staff memberships and a complete operational metrics dashboard are not implemented. The same account may own registries and one business.

Document map

Pages 2 and 3 describe journeys and company responsibilities. Pages 4 and 5 define user stories. Pages 6 through 10 describe the database. Pages 11 and 12 cover tokens and security. Pages 13 and 14 establish efficiency and scaling rules. Page 15 provides release gates and references. The editable companion includes the exact Prisma schema.

Application flow for owners and guests

Account entry and onboarding

A visitor chooses Haitian Creole, French or English. Google sign-in works locally for the configured test account. Microsoft, GitHub and email sign-in are offered only when configured; their hosted journeys remain pending. Auth.js validates the external callback, resolves the account and creates a session. The welcome flow can save profile and language details, select an occasion, or defer onboarding. A supplied valid internal callback remains important when returning to a task.

Create and maintain a registry

The owner selects baby shower, wedding, birthday, Christmas, shopping or other; enters a title, optional description and date; and saves a draft. Blank optional links must remain valid. Gifts can be manual entries, external store links or products from approved businesses. The bulk catalog picker supports up to 20 distinct selections in one atomic batch. A failed batch adds none of its gifts.

The owner reviews items and explicitly publishes a nonempty list. Guests can open only PUBLISHED, public, unprotected lists. DRAFT lists remain hidden through status eligibility even though isPublic defaults to true in the schema. ARCHIVED lists are unavailable publicly. A password field exists, but a complete protected-list unlocking journey is not established.

Share and find a list

The owner shares the exact registry link or downloads a QR share card containing the list title, occasion, date when available, scanning instruction and readable address. Guests can scan a saved image locally, request camera scanning, or paste the link/exact address. Only same-origin registry destinations are accepted. Search by a friend name or event title is not currently implemented. Registry occasion types and merchant product categories are different concepts.

Choose and pay when activated

The guest selects eligible linked merchant gifts and quantities. The server rechecks list eligibility, approved merchant status, current USD price and physical stock. It creates the order and reserves both registry quantities and shared product units in one transaction. Stripe session creation happens after that transaction. No browser price is authoritative.

Confirm and recover

A signed paid provider event must match the order, amount, currency and session. Payment state, order state and held stock consumption commit together, once. Email is attempted after commit. The return page is not proof of payment. Duplicate events, lost responses and reloads must reuse the existing order; provider-confirmed expiry releases unpaid holds. Ambiguous errors retain them for reconciliation. Paid does not mean delivered.

Company merchant and administrator flow

Register a merchant

The applicant signs in and submits business name and category, with optional contact, location and patente number. The patente is a bounded string preserving leading zeroes and printed separators. It is visible only to its owner and reviewing administrators. Capturing a number does not verify government registration. One business per account is enforced by Business.ownerId uniqueness.

Review and publish products

Registration creates PENDING_REVIEW and assigns BUSINESS_OWNER unless the actor is already ADMIN. Applicants may maintain products while pending; public listing and native checkout require APPROVED. An administrator approves or suspends the business. SUSPENDED merchants cannot change products, and their products are excluded from public eligibility. Approval criteria, evidence retention and an immutable review audit trail remain operational work.

Maintain the catalog

The merchant sets product details, an actual price and physical stock. stockQuantity is the unsold count including checkout holds; reservedStock is its held subset. Blank stock is unknown, not unlimited, and prevents native payment. The availability switch can disable sales independently. Active checkouts block product edits. A gift being added to a registry does not reserve stock. Catalog price changes do not silently reprice existing gifts.

Operate after payment

A merchant paid-order list exists. Accepting, packing, dispatching, delivering, handling returns and settling with vendors do not yet form a complete operating workflow. The company must assign responsibility for each state change, exception and customer message. A commissionRate field defaulting to 5 does not implement commissions or settlement. Current native checkout uses zero platform fee.

Responsible party	Required accountability before live payments
Bayou operating company	Name the legal entity and receiving country; choose seller and settlement model; publish delivery, cancellation, refund and support terms; appoint incident and reconciliation owners.
Merchant	Confirm physical inventory, price, product quality and delivery or pickup capability; maintain accurate counts and evidence of handover.
Administrator	Review applicants under documented criteria, restrict sensitive access, record decisions and escalate suspicious or disputed activity.
Provider and finance operator	Confirm provider eligibility and destination account; reconcile gross payments, fees, refunds, disputes and vendor liabilities.

No legal entity details, banking eligibility, marketplace payout contract or tax treatment are established by this document. These are explicit business decisions, not implied capabilities of an APPROVED merchant record.

User stories for registry owners and givers

US01 Start without losing my task

As a new user, I want to sign in and complete or skip onboarding so I can return to the task I started. Acceptance: preserve a valid internal callback, prefill available profile details, persist progress, reject external redirects and show a useful cancelled-login state. Status: implemented locally; hosted providers pending.

US02 Make a useful private draft

As a registry owner, I want to choose an occasion, title and date and save a draft before sharing. Acceptance: optional blank links do not crash; invalid fields appear together and receive focus; inputs survive failure; only the owner can edit; guests cannot open the draft. Status: implemented.

US03 Add several gifts at once

As a registry owner, I want to select several approved catalog gifts and add them together. Acceptance: at most 20 distinct products, a reviewable selection, no duplicate entries, authoritative prices, and complete rollback if any selection becomes unavailable. Status: implemented with database tests.

US04 Share something people understand

As an owner, I want a link and a downloadable QR card so relatives know which list they are opening. Acceptance: title, occasion, optional date and readable destination appear around a decodable QR code; the quiet zone remains clear; nonpublic lists cannot generate public share cards. Status: implemented.

US05 Find a shared registry

As a guest, I want to scan a shared QR or paste the exact list link so I reach the intended gifts. Acceptance: validate the current site, preview the destination, reject unsafe or external links, stop camera tracks and provide image/manual fallback. Status: implemented; physical-camera verification pending. Name/event search is a separate planned feature requiring owner opt-in.

US06 Purchase once despite a bad connection

As a giver, I want retries and reloads to resume my original checkout so I do not reserve or pay twice. Acceptance: one attempt maps to one order; changed details cannot reuse it; the held last unit can be resumed; the original signed-in buyer is required when bound; only verified provider evidence confirms success. Status: code and database tests implemented; real-provider journey pending and payments disabled.

US07 Read honest availability

As a giver, I want to distinguish a confirmed gift from an unpaid hold so I do not mistake a reservation for a purchase. Acceptance: confirmed fulfillment and remaining quantity derive from stored accounting; a pending payment never increases confirmed progress. Status: implemented at registry and checkout accounting layers.

User stories for merchants administrators and future baskets

US08 Apply as a merchant

As a business owner, I want to submit my company details and patente number for review. Acceptance: one application per account, pending status, no self-approval, private patente display, safe validation, and preserved input with visible errors. Status: core workflow implemented; complete hosted merchant journey pending.

US09 Control real stock

As a merchant, I want to enter physical quantities and see held units so two registries cannot sell my last product. Acceptance: nonnegative whole counts, unknown stock blocks payment, cross-owner edits fail, active holds block edits, concurrent buyers share one count, and expiry restores only unpaid holds. Status: implemented with PostgreSQL concurrency tests; full merchant browser journey pending.

US10 Review businesses

As an administrator, I want to approve or suspend a merchant so public discovery and checkout reflect the review outcome. Acceptance: use current database role, no cross-role elevation, suspended products cannot be purchased, and sensitive identifiers stay outside public DTOs. Status: basic review implemented; durable decision audit and broader operations pending.

US11 Monitor and resolve orders

As the operating company, I want dashboards for payment failures, aged holds, delivery exceptions, refunds and balances so I can resolve customer issues. Acceptance: role-scoped views, defined metric meanings, reconciliation traceability, no customer data in public analytics and audited financial adjustments. Status: planned. A paid-order list does not satisfy this story.

US12 Build a basket for someone else

As a gift organizer, I want to curate gifts for a recipient and choose whether I buy the basket or share it with others. Acceptance: the two modes are explicit at creation; saving starts as a private draft; no recipient account or automatic notification is created; currencies stay separate; approved catalog items use server prices; external gifts remain external purchases. Status: approved scope, not implemented.

US13 Personalize and finish my basket

As an organizer, I want a style preview, quantity review and clear sharing settings so the basket reflects the occasion without changing account access. Acceptance: accessible theme selection, persistent presentation choices, owner-only mutation, no automatic public name discovery, retained values on failure and supported HT/FR/EN copy. Status: planned with US12; visitor light/dark preference already exists.

US14 Recover safely from a failure

As an operator, I want verified backups, alerts and a reversible deployment process so an outage does not corrupt paid history. Acceptance: restore into a separate database, prove access and reconciliation, retain signed-event recovery, and document application rollback independently of data repair. Status: release requirement, not yet demonstrated.

Database structure and lifecycle rules

PostgreSQL is the source of truth, accessed through Prisma 7.10.1 and its PostgreSQL adapter. The 11 models separate identity, catalog, registry intent and payments. The next pages inventory their fields; the companion includes exact Prisma defaults, relations and indexes.

Relationship	Cardinality and enforcement
User to Account and Session	One user to many provider accounts and optional session rows; JWT sessions are active.
User to Registry and Business	Many registries per user; at most one business per owner.
Business to Product	One business to many products; each product has one merchant.
Registry to RegistryItem	One registry to many gift entries. A local product may appear on several registries.
Order to OrderItem and Payment	Many lines and at most one payment per order. Buyer may be a user or guest.
OrderItem to RegistryItem and Product	Lines reference registry items and snapshot productId with a restrictive FK. Historical productId may be null.

Important relationship limitation

RegistryItem.productId is currently a nullable string without a Prisma product relation or database foreign key. It is an application-level catalog reference. Checkout revalidates it. Do not mistake it for the stronger OrderItem.productId foreign key. Future cleanup or migration must account for orphaned draft/catalog references.

Current states

Registry: DRAFT, PUBLISHED, ARCHIVED. Business: PENDING_REVIEW, APPROVED, SUSPENDED. Item: AVAILABLE, PARTIALLY_FULFILLED, FULFILLED. Order: PENDING, PAID, CONFIRMED, SHIPPED, DELIVERED, CANCELLED, REFUNDED. Payment: PENDING, PROCESSING, COMPLETED, FAILED, REFUNDED. Later order and refund statuses exist in the schema without a complete controlled workflow.

Inventory invariants

A registry quantity is positive; fulfilled and reserved quantities are nonnegative; their sum cannot exceed the requested quantity. Physical reserved stock is nonnegative and cannot exceed confirmed physical stock. Unknown physical stock requires zero reserved stock. Confirmed payment consumes both physical and registry holds once. Refunds do not automatically restock units.

Money and history

Stored money uses Decimal precision 12 scale 2. Calculations use integer minor units. Native checkout currently charges USD only, with explicit Payment.currency. Never write an estimated HTG conversion as a paid transaction amount. Archive registries and preserve order references; do not perform blind account or merchant cascade deletion against financial history.

Database dictionary for identity and authentication

Notation: PK is primary key, UQ is unique, FK is foreign key, and nullable means optional storage. IDs are application-generated CUID strings unless otherwise stated. DateTime is used for stored timestamps. User-controlled business identifiers are not authentication credentials.

Model and table	Stored fields and constraints
User users	id PK; email required UQ; emailVerified nullable; name, image and phone nullable; preferredLocale default ht; onboardingStep default 0; onboardingCompletedAt and onboardingDeferredAt nullable; role default USER; createdAt and updatedAt.
Account accounts	id PK; userId FK; type, provider and providerAccountId required; refresh_token, access_token and id_token nullable Text; expires_at nullable Int; token_type, scope and session_state nullable strings. Unique provider plus providerAccountId; userId index.
Session sessions	id PK; sessionToken UQ; userId FK and index; expires required DateTime. Schema support remains present, but active authentication uses JWT sessions rather than database session lookup as its primary strategy.
VerificationToken verification_tokens	identifier required; token UQ; expires required DateTime. Compound uniqueness on identifier plus token. This model has no separate id. Used for framework-managed verification flows when email sign-in is configured.

Role boundaries

USER can manage owned registries and buy gifts. BUSINESS_OWNER additionally manages the business associated with its own ownerId. ADMIN can access implemented review operations after a current database role check. A role claim alone does not authorize access to another owner record. The proxy is a navigation hint; services enforce ownership and privilege.

Token storage limitation

Provider token columns are ordinary nullable database text in the current schema. Application-level field encryption and refresh-token rotation orchestration are not established by these columns. Minimize requested provider scopes, protect database access and backups, and implement encrypted-at-rest token storage if provider API access is retained beyond sign-in.

Account linking and deletion

Do not automatically link separate provider accounts solely because emails match. Account and Session user relations cascade on deletion, while user-owned registries and business records also cascade through their relations. Financial references can block deletion. A deliberate redaction/archive process is required before offering account deletion that must preserve payment history.

Identity evidence

The Account provider mapping identifies a social login. It is separate from merchant approval and from legal company verification. Email verification, a business role, a submitted patente and a completed purchase prove different facts and must not be treated as substitutes for each other.

Database dictionary for registries and gift entries

Model and table	Stored fields and constraints
Registry registries	id PK; slug UQ; userId FK; type required RegistryType; status default DRAFT; title required; description nullable Text; eventDate nullable DateTime; coverImage nullable; locale default ht; ogImageUrl nullable; amazonWishlistUrl nullable; externalLinks nullable Json; isPublic default true; password nullable; createdAt and updatedAt.
RegistryItem registry_items	id PK; registryId FK; title required; description nullable Text; imageUrl nullable; priceHTG and priceUSD nullable Decimal 12,2; quantity default 1; fulfilledQty and reservedQty default 0; status default AVAILABLE; source default CUSTOM; externalUrl, businessId and productId nullable; priority default 0; createdAt and updatedAt.

Types and sources

RegistryType is BABY_SHOWER, WEDDING, BIRTHDAY, CHRISTMAS, SHOPPING or OTHER. ItemSource is CUSTOM, LOCAL_BUSINESS, AMAZON or EXTERNAL. A custom/manual entry or external store link is list intent, not a cash fund or a Bayou-payable merchant product. A native checkout requires linked approved merchant inventory.

Publication and privacy

Public readers require status PUBLISHED, isPublic true and no password. Public transfer objects omit credentials and internal hold counts. An owner may see management data, but public progress must distinguish confirmed fulfillment from unavailable units. Password-protected lists remain hidden until a secure, complete unlocking flow is implemented; the password column is not evidence of a password hashing or verification implementation.

Indexes

Registry indexes support userId, slug, status plus isPublic plus createdAt, and userId plus status plus updatedAt. RegistryItem indexes cover registryId and businessId. Slug is already unique; its additional ordinary index should be evaluated for redundancy through query plans before changing it.

Database checks

The migration enforces quantity greater than zero, nonnegative fulfilledQty and reservedQty, and fulfilledQty plus reservedQty no greater than quantity. Each stored price is either null or positive. Keep constraints in migrations as well as application validation; some checks are not expressible in Prisma schema syntax.

Retention and safe edits

RegistryItem has order references. Items with reservations or purchase history cannot be casually removed or rewritten. Archive a registry rather than destroying order context. Preserve the stored price snapshot on an existing order even if a merchant later changes its catalog. Slugs and QR links should survive display-brand changes.

Database dictionary for merchants and products

Model and table	Stored fields and constraints
Business businesses	id PK; ownerId UQ and User FK; name required; slug UQ; description nullable Text; logoUrl and coverUrl nullable; category required; patenteNumber nullable VarChar 100; city, department, address, phone, whatsapp, email and website nullable; moncashNumber, natcashNumber and stripeAccountId nullable; status default PENDING_REVIEW; commissionRate Decimal 4,2 default 5.0; createdAt and updatedAt.
Product products	id PK; businessId FK; name required; description nullable Text; imageUrl nullable; priceHTG and priceUSD nullable Decimal 12,2; inStock default true; stockQuantity nullable Int; reservedStock default 0; category nullable; createdAt and updatedAt. Referenced by order lines.

Company identifier privacy

patenteNumber is an identifier, not a numeric quantity. Preserve leading zeroes and safe printed separators. It is optional and private to the owner and administrators. No government lookup, validity certification or automatic approval is implied. Contact and payout fields require a documented disclosure policy before exposing them publicly.

Inventory meaning

stockQuantity represents unsold physical units including units held during checkout. Available units are stockQuantity minus reservedStock, provided the merchant has enabled the product. Null means unconfirmed. A zero count means sold out. Several registries reference the same count; adding a product to a wishlist does not reduce it.

Constraints and indexes

Positive-or-null prices are enforced by database checks. The stock check requires reservedStock nonnegative and within stockQuantity; null stock requires zero holds. Business indexes cover slug and status plus name. Product indexes cover businessId and inStock plus name plus id. Checkout locks products in a consistent order before conditional updates.

Payment and deletion boundaries

moncashNumber, natcashNumber and stripeAccountId are configuration/contact fields, not proof of a working provider integration. commissionRate is not an implemented payout ledger. Business products cascade on business deletion, but an OrderItem product foreign key uses RESTRICT, preserving referenced inventory history. Deletion plans must respect both behaviors.

Future company model

Multi-staff businesses require a separate CompanyMember or BusinessMember relation with scoped roles, invitation lifecycle and audit records. Do not grant every staff member BUSINESS_OWNER or expose one merchant data through another account. This relation is proposed and does not exist in the current schema.

Database dictionary for orders and payments

Model and table	Stored fields and constraints
Order orders	id PK; orderNumber UQ; checkoutRequestId nullable UQ; checkoutRequestHash and checkoutLocale nullable; buyerId nullable User FK; buyerName and buyerEmail required; buyerPhone nullable; giftMessage nullable Text; isAnonymous default false; totalHTG, totalUSD and platformFee nullable Decimal 12,2; status default PENDING; createdAt and updatedAt.
OrderItem order_items	id PK; orderId FK; registryItemId FK; productId nullable Product FK with RESTRICT deletion; quantity default 1 and greater than zero; priceHTG and priceUSD nullable Decimal 12,2. Indexes on orderId, registryItemId and productId.
Payment payments	id PK; orderId UQ FK; provider required; status default PENDING; stripePaymentId, paypalOrderId, moncashTxId and natcashTxId nullable; checkoutUrl nullable Text; providerEventId nullable UQ; amountHTG and amountUSD nullable Decimal 12,2; currency required; providerResponse nullable Json; paidAt nullable; createdAt and updatedAt.

Provider and state enums

PaymentProvider contains STRIPE, PAYPAL, MONCASH, NATCASH and EXTERNAL_REDIRECT. Enum membership does not enable payment. Only the guarded Stripe path has the current native purchase implementation. Order states after PAID and refund states require further operating workflows. Order indexes cover buyerId, orderNumber and status plus createdAt; Payment indexes cover stripePaymentId and moncashTxId.

Retry identity

checkoutRequestId is a random browser attempt capability. A hash binds canonical selection, quantities, buyer identity/details and locale. A transaction advisory lock and unique key reuse the same order on identical retries. A changed request cannot reuse it. checkoutLocale preserves provider return routing. checkoutUrl is private and used to resume the saved provider session.

Payment evidence and idempotency

providerEventId uniqueness and a conditional PENDING order claim prevent repeat fulfillment. There is no complete webhook-event inbox or general IdempotencyKey log table. Payment.providerResponse must not become an unrestricted store or public dump of provider payloads. OrderItem productId is populated for new native orders but may be null on historical records.

Current financial limits

Native checkout charges USD from 50 cents through 999999 cents and stores zero platform fee. It does not implement tax/shipping calculation, group contribution balances, split payments, commissions, vendor settlement or partial refunds. Multi-merchant order lines exist without a complete merchant allocation and delivery model.

Security token inventory and handling

These are token types and configuration names, not secret values. Each has a different trust boundary. Public item IDs, a patente number and a list slug are never substitutes for authentication. Token values must not appear in this document, Git, analytics, screenshots of credentials or ordinary logs.

Token or secret	Current purpose and storage	Required handling
Auth.js session	Encrypted JWT session in framework cookie; configured lifetime 24 hours. Current role re-read from database during session resolution.	HttpOnly and SameSite Lax framework defaults; Secure on HTTPS. Keep off localStorage. Test hosted cookie/host behavior.
AUTH_SECRET	Server configuration used by Auth.js to protect tokens.	Generate securely; separate environments; rotate under an incident procedure. Never NEXT_PUBLIC or client props.
OAuth client secret	Google and optional provider server configuration.	Exact approved callback origins; least provider scopes; no automatic email-based account linking.
OAuth access and refresh tokens	Nullable Account text columns managed through the adapter.	Private server/database access only. Field encryption and a broader provider-token lifecycle remain work.
Email verification token	Framework-managed VerificationToken; email provider configured for 15-minute links.	Expiry and one-time framework consumption; verify single use and delivery when enabled.
OAuth and CSRF checks	Framework-managed callback/request protections.	Do not replace with userId fields. Verify actual provider checks, origin validation and proxy headers.
Checkout attempt UUID	Random capability in sessionStorage, with memory fallback; uniquely mapped to Order.	Not an account session. Bind registry and original signed-in buyer. Return only provider destination, not private order data.
STRIPE_WEBHOOK_SECRET	Server-only signature verification over the raw request body.	Do not trust a success URL. Validate stored amount, currency and provider session before fulfillment.
CRON_SECRET and service keys	Bearer protection for reconciliation; Redis, storage, sender and provider keys stay server-side.	Separate staging/production, minimal access, redact logs and test invalid/missing credentials.

The public QR contains an eligible registry URL, not a bearer credential for editing or a payment authorization. The CSP nonce permits trusted scripts for one rendered response; it is not a user identity token. Appearance cookies and retained legacy key names are compatibility settings, not authentication secrets.

Security rules and current gaps

Authentication and authorization

Resolve the server session and current user/role for every sensitive operation. Constrain each query and mutation by owner or explicitly authorized administrator. Deny access by default. Proxy navigation checks are insufficient for authorization. These rules follow OWASP authorization guidance [R1]. ADMIN has no separate superadmin/MFA workflow in this baseline; strong provider authentication and admin audit controls are release requirements.

Inputs outputs and browser defenses

Parse runtime input with strict schemas; reject unexpected fields, duplicate form keys, bad quantities and unsafe URLs. Use parameterized database operations. Render user text as text and escape email HTML. Production scripts use a nonce CSP; frame embedding and objects are blocked. Styles still allow inline CSS, images allow HTTPS sources, and CSP alone does not prove XSS safety. Audit uploads, imported links and error rendering independently.

Session and secret lifecycle

Use HTTPS end to end and preserve framework cookie defenses. Never store account sessions in browser storage. Rotation, revocation and idle/absolute lifetime behavior need hosted verification. A 24-hour JWT is not an implemented per-device revocation registry. Auth.js token encryption does not encrypt Account token columns. Recommendations on cookie isolation and token lifetime follow OWASP session guidance [R2].

Rate limits and abuse

Configured one-minute buckets: authentication 10, checkout 10, general 60 and webhook 300. Production fails closed when Redis is missing or unavailable. The proxy uses a host-aware trusted IP helper. The checkout action also reads x-forwarded-for for its extra guest bucket; normalize it to the same trusted helper before claiming uniform spoof-resistant protection. Size, timeout and upload limits must be exercised on the actual host.

Payments and sensitive data

Verify signed raw-body provider events and stored payment evidence; commit fulfillment only once. Keep all payable inventory decisions uncached. Never release a still-payable session based on a browser timer. Notifications are best-effort after payment commit. No card data should traverse Bayou application forms. A complete event inbox, retryable notification outbox, refund ledger and seller settlement accounting remain planned. Stripe fulfillment guidance supports provider-confirmed, repeat-safe fulfillment [R3].

Release gaps requiring explicit tests

Prove cross-account and stale-role rejection, host/header trust, CSRF/origin behavior, stored XSS cases, blocked unsafe URLs, Redis outage response, upload ownership/type/size limits, secret-free bundles/logs and backup isolation. Add a durable admin audit trail, verified retention policy and tested incident/revocation procedures. A clean dependency audit is evidence at a point in time, not a security certification.

Efficiency and database performance rules

Rule	Current behavior and engineering requirement
Bound every list	Directory uses 12-item keyset pages with stable name/id ordering. Public product reads have caps and a 60-second catalog cache. Owner/admin lists still need pagination review as data grows. Never load the full merchant catalog into the homepage.
Keep authoritative reads fresh	Cache eligible public presentation only. Re-read prices, merchant approval, stock, sessions and order evidence at mutation time. Invalidate public catalog tags after edits. A cached availability label is never a purchase guarantee.
Keep writes atomic	Use conditional increments under locks and database checks. Reserve registry and shared product stock with order creation. Acquire SKU locks in a consistent order. Network calls and email stay outside the database transaction.
Control connections	Current application pool: maximum 4 per process, 5-second connect timeout, 10-second idle timeout. Total potential connections grow with active application instances; use a hosted pooled URL and provider-specific budgets.
Measure before adding indexes	Use representative staging data and EXPLAIN ANALYZE for hot queries. Composite indexes should match filtering and ordering. Substring name/city search can become expensive; evaluate a targeted search index only after measurement.
Reduce transfer and rendering	Select explicit DTO fields, retain bounded pages, compress/resize merchant images and avoid N+1 reads. Keep fonts local and defer optional code. Preserve touch scrolling, reduced motion and background pause behavior.
Budget animation	Current stars are bounded and pause when hidden; no decorative WebGL is required for core tasks. The original 3D budget is less than 150 kB gzip additional JavaScript. Any future 3D must lazy-load and retain a usable flat fallback.

Acceptance targets rather than measured guarantees

Target LCP at most 2.5 seconds, INP at most 200 milliseconds and CLS at most 0.1 at the 75th percentile, with meaningful mobile and constrained-network tests. Establish API and database baselines on the hosted environment before setting p95 latency SLOs. Record payload size, query count, lock wait, pool saturation and error rate with each load test.

Rule for optimization work

Do not trade away ownership checks, provider verification, stock constraints or correctness to improve a benchmark. Optimize the measured bottleneck, prove the invariants still hold, and stop adding caching or infrastructure when the evidence no longer supports it.

Scalability hosting and recovery rules

Pilot architecture

Keep a modular Next.js application with thin actions and service-owned business logic, PostgreSQL as the source of truth, shared Redis for limits, object storage for images and a verified email sender. Remaining raw action queries should move behind services. Do not introduce microservices before a measured operational need. The target free-service plan is Netlify, Neon, Upstash and R2; this document does not assert those accounts are provisioned or their current quotas guaranteed.

Annual budget and deployment constraints

The owner cap is 50 USD total for year one. Use free services first, spending alerts, hard usage controls where supported and no automatic paid upgrades. Reserve domain spending until exact registration and renewal costs are approved. Existing Vercel integration can create previews after pushes. The current five-minute Vercel cron was rejected on Hobby; it is not an accepted free-host deployment design.

Scale the stateless application carefully

Share database and limiter state across instances; do not depend on local memory for production rate limiting or durable orders. Keep image bytes outside PostgreSQL. Separate staging data/secrets from production. With a four-connection per-instance pool, cap instance concurrency against the total database connection budget. Co-locate app and database regions where feasible and measure cold starts and retry behavior.

Introduce durable background work when needed

Use a durable signed-event inbox and notification outbox before increased throughput requires reliable retry and backpressure. Consumers must be idempotent under at-least-once delivery. Track attempt count, next retry, processed timestamp and terminal failure handling. These tables/queues are proposals, not current schema. Keep provider operations bounded and never acknowledge an uncommitted paid transition.

Recovery and reconciliation

The source contains an optional Netlify reconciliation function, while its schedule and enabled configuration still require validation. Webhooks remain primary. Select a scheduler that fits the actual free plan and maximum acceptable hold age. Monitor oldest pending order, failed reconciliation, webhook failures, lock waits, pool use, error rate and service quotas. Test encrypted off-database backups and restoration into a separate environment; choose and prove recovery objectives before launch.

Scale decisions based on triggers

If catalog search becomes slow, inspect query plans before adding search infrastructure. If provider traffic backs up, introduce a durable queue with bounded consumers. If connection waits rise, tune pooling and instance concurrency before adding database replicas. If read replicas are later introduced, keep payment/stock decisions on the authoritative writer. App rollback must remain compatible with migrations and preserve paid history.

Acceptance gates decisions and source references

Gate	Evidence needed before moving forward
Registry and merchant journeys	HT/FR/EN owner create/edit/publish/share; foreign-user denial; QR decode/scan; merchant review and stock editing; correct empty, failed and pending states on desktop and mobile.
Inventory and retry safety	Two registries compete for the last SKU; one reservation wins. Test multi-line rollback, duplicate/late events, pay-versus-expiry, unknown stock, changed price, buyer tampering and reload resume.
Provider and finance	Eligible real seller/provider account; Stripe test success/decline/cancel/expiry/delayed webhook; manual refund and actual settlement evidence; agreed tax/shipping and merchant payout model.
Hosted security and recovery	Verify HTTPS/cookies/origin trust, public/private DTOs, Redis refusal/outage, upload policies and redacted monitoring. Complete a separate-database restore and rehearse checkout disable/recovery.
Controlled pilot	No seeded merchant presented as real stock; verified Google hosted branding/callback; reconciler fits the chosen plan; named support/incident owners; budget alerts and rollback procedure.

Recorded code evidence

The rename checkpoint records 242 passing unit tests, 23 PostgreSQL integration tests, TypeScript and production-build success. Payment integration tests mock Stripe at the network boundary. These are recorded baseline results, not a new test run or a complete hosted/security approval for this document. Browser branding and an existing local Google session were verified. Live payment remains disabled.

Next implementation order

Implement the approved two-mode recipient basket, then merchant fulfillment/refund operations, operating dashboards and durable audit/recovery. Settle legal entity, seller, delivery, settlement and refund responsibilities before enabling money collection. Complete hosted provider and recovery gates before a controlled launch. Add opt-in registry name/event discovery only with explicit privacy rules.

Repository references

Baseline: github.com/AlexandreC1/kadolakay at f3167e8. Core evidence: prisma/schema.prisma and migrations; src/lib/auth.ts; src/services/authorization.ts; src/proxy.ts; src/lib/security.ts; src/services/checkout.ts and inventory.ts; src/lib/orders.ts; src/app/api/webhooks/stripe/route.ts; catalog services; docs/HANDOFF.md, FEATURE-STATUS.md, BASKET-FEATURE.md and FREE-HOSTING.md. Other branches or PRs may differ and must be compared before combining work.

[R1] OWASP Authorization Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

[R2] OWASP Session Management Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

[R3] Stripe Checkout fulfillment. https://docs.stripe.com/checkout/fulfillment